

T.C.  
ERCIYES ÜNİVERSİTESİ  
MÜHENDİSLİK FAKÜLTESİ  
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ



**MOBILE APPLICATION DEVELOPMENT DERSİ  
PROJE RAPORU**

**DART ASENKRON PROGRAMLAMA**

SEMİH ÇAY  
1030510327

DR. ÖĞRETİM ÜYESİ FEHİM KÖYLÜ

NİSAN 2025  
KAYSERİ

# Dart Asenkron Programlama Raporu

## Giriş

Modern uygulamalarda kullanıcı deneyimini bozmadan işlem yapmak için **asenكرون programlama** büyük önem taşır. Dart dili, özellikle Flutter ile mobil uygulama geliştirirken arka planda zaman alan işlemleri (ağ istekleri, dosya okuma/yazma, gecikmeler vb.) asenkron olarak gerçekleştirmeyi sağlar.

## Senkron ve Asenkron Programlama Arasındaki Fark

| Özellik        | Senkron                            | Asenkron                              |
|----------------|------------------------------------|---------------------------------------|
| Çalışma Sırası | İşlemler sırayla yapılır           | İşlemler aynı anda başlatılabilir     |
| Bekleme        | Zaman alan işlem diğerini bekletir | Zaman alan işlem beklemeden yürütülür |
| Performans     | Düşebilir                          | Daha verimlidir                       |

## Dart'ta Asenkron Yapılar

Dart dilinde asenkron işlemleri gerçekleştirmek için 3 temel yapı kullanılır:

### Future

**Future**, gelecekte bir değeri temsil eder. Zaman alan işlemler sonucunda bir değer döner.

```
Future<String> fetchData() async {  
  await Future.delayed(Duration(seconds: 2));  
  return 'Veri başarıyla alındı!';  
}
```

### async ve await Anahtar Kelimeleri

- async**: Bir fonksiyonun asenkron olduğunu belirtir.
- await**: Asenkron işlemin sonucunu bekler.

```
void main() async {  
    print('Veri alınıyor...');  
    String veri = await fetchData();  
    print(veri);  
}
```

## then() Metodu

Alternatif olarak `Future` 'ın tamamlandığında çalışmasını istediğimiz işlemi `then()` ile tanımlayabiliriz:

```
fetchData().then((veri) {  
    print(veri);  
});
```

---

## Stream Nedir?

`Stream`, zamanla birden fazla veri üretir. Örneğin sensör verileri, kullanıcı etkileşimleri, canlı güncellemeler gibi.

```
Stream<int> sayici() async* {  
    for (int i = 0; i < 5; i++) {  
        await Future.delayed(Duration(seconds: 1));  
        yield i;  
    }  
}
```

Kullanımı:

```
await for (var sayi in sayici()) {  
    print('Sayı: $sayi');  
}
```

---

## Uygulama Örneği: Gecikmeli Veri Getirme

```
Future<void> veriGetir() async {  
    print('Veri getiriliyor...');
```

```
    await Future.delayed(Duration(seconds: 3));  
    print('Veri başarıyla getirildi!');  
}  
  
void main() {  
    veriGetir();  
    print('Diğer işlemler devam ediyor...');  
}
```

### Çıktı:

```
Veri getiriliyor...  
Diğer işlemler devam ediyor...  
Veri başarıyla getirildi!
```

Bu çıktı, asenkron yapının nasıl arka planda çalıştığını gösterir.

---

## Sonuç

Dart'ta asenkron programlama, uygulamaların donmadan, akıcı şekilde çalışabilmesi için gereklidir. `Future`, `async`, `await` ve `Stream` yapıları, geliştiricilere zaman alan işlemleri etkili biçimde yönetme imkanı sunar. Özellikle Flutter uygulamalarında ağ istekleri ve kullanıcı deneyimi açısından büyük avantaj sağlar.